



UNIVERSIDAD
NACIONAL
DE LA PLATA

SISTEMAS OPERATIVOS

Práctica 5 - Seguridad (actualizada)

Notas:

1. **Se recomienda trabajar en una VM** ya que como parte de la práctica se van a habilitar y deshabilitar medidas de seguridad, lo que puede generar vulnerabilidades o hacer que determinadas aplicaciones no funcionen.
2. Utilizar un kernel completo (no el compilado en las prácticas 1 y 2) En el caso de la VM de la cátedra se puede usar el **kernel que viene por defecto (6.1.0)**.
3. Compilar el código C **usando el Makefile provisto** (el makefile deshabilita algunas medidas de seguridad del compilador y generar un código assembler más simple).
4. Acceda al código necesario para la práctica en el repositorio de la materia.
5. En Debian 12 (Woodworm) utilizar el kernel por defecto 6.1.0 para evitar incompatibilidades con apparmor-utils.

A - Introducción

1. Defina política y mecanismo.
2. Defina objeto, dominio y right.
3. Defina POLA (Principle of least authority).
4. ¿Qué valores definen el dominio en UNIX?
5. ¿Qué es ASLR (Address Space Layout Randomization)? ¿Linux provee ASLR para los procesos de usuario? ¿Y para el kernel?
6. ¿Cómo se activa/desactiva ASRL para todos los procesos de usuario en Linux?

B - Ejercicio introductorio: Buffer Overflow simple

El propósito de este ejercicio es que las y los estudiantes tengan una introducción simple a un stack buffer overflow a fin de poder abordar el siguiente ejercicio. Las y los estudiantes aprenderán a identificar la vulnerabilidad, analizar la disposición de la memoria y construir una entrada que aproveche la vulnerabilidad para obtener acceso no autorizado a una función privilegiada.

Nota: Puede ser de ayuda ver el código assembler generado al compilar (00-stack-overflow.s) o utilizar gdb para depurar el programa pero no es obligatorio.

1. Compilar usando el makefile provisto el ejemplo 00-stack-overflow.c provisto en el repositorio de la cátedra.
2. Ejecutar el programa y observar las direcciones de las variables `access` y `password`, así como la distancia entre ellas.
3. Probar el programa con una password cualquiera y con "big secret" para verificar que funciona correctamente.
4. Volver a ejecutar pero ingresar una password lo suficientemente larga para sobrescribir `access`. Usar `distance` como referencia para establecer la longitud de la password.
5. Después de realizar la explotación, reflexiona sobre las siguientes preguntas:
 - a. ¿Por qué el uso de `gets()` es peligroso?
 - b. ¿Cómo se puede prevenir este tipo de vulnerabilidad?
 - c. ¿Qué medidas de seguridad ofrecen los compiladores modernos para evitar estas vulnerabilidades?

C - Ejercicio: Buffer Overflow reemplazando dirección de retorno

Objetivo: El objetivo de este ejercicio es que las y los estudiantes comprendan cómo una vulnerabilidad de desbordamiento de búfer puede ser explotada para alterar la dirección de retorno de una función, redirigiendo la ejecución del programa a una función privilegiada. Además, se explorará el mecanismo de seguridad ASLR y cómo desactivarlo temporalmente para facilitar la explotación.

Nota: Puede ser de ayuda ver el código assembler generado al compilar (`01-stack-overflow-ret.s`) o utilizar gdb para depurar el programa pero no es obligatorio.

1. Compilar usando el makefile provisto el ejemplo 01-stack-overflow-ret.c provisto en el repositorio de la cátedra.
2. Configurar `setuid` en el programa para que al ejecutarlo, se ejecute como usuario `root`.
3. Verificar si tiene ASLR activado en el sistema. Si no está, actívelo.
4. Tenemos los siguientes datos sobre el stack:
 - a. El stack crece hacia abajo.
 - b. Si estamos compilando en `x86_64` los punteros ocupan 8 bytes.
 - c. `x86_64` es little endian.
 - d. Primero se apiló la dirección de retorno (una dirección dentro de la función `main()`). Ocupa 8 bytes.
 - e. Luego se apiló la vieja base de la pila (`rbp`). Ocupa 8 bytes.
 - f. Luego se apila `debug` que ocupa 1 byte pero puede tener bytes de relleno (verificar en el archivo `.s` la cantidad de padding que se agrega entre `debug` y `password`).

g. `password` ocupa 16 bytes.

Calcule cuántos bytes de relleno necesita para pisar primero `debug` y luego la dirección de retorno.

5. Ejecute `01-stack-overflow-ret` al menos 2 veces pisando el valor de `debug` para verificar que la dirección de memoria de `privileged_fn()` cambia.
6. Apague ASLR y repita el punto 3 para verificar que esta vez el proceso siempre retorna la misma dirección de memoria para `privileged_fn()`.
7. Ejecute el script `generate_payload.py` para generar el payload.
8. Pruebe el payload redirigiendo la salida del script a `01-stack-overflow-ret` usando un pipe.
9. Para poder interactuar con el shell modifique `exploit.expect` para que envíe el payload correcto y ejecute ese archivo con el comando `expect`.
10. Pruebe algunos comandos para verificar que realmente tiene acceso a un shell con UID 0.
11. Conteste:
 - a. ¿Qué efecto tiene setear el bit `setuid` en un programa si el propietario del archivo es `root`? ¿Qué efecto tiene si el usuario es por ejemplo `nobody`?
 - b. ¿Cómo ASLR ayuda a evitar este tipo de ataques en un escenario real donde el programa no imprime en pantalla el puntero de la función objetivo?
 - c. ¿Cómo podría evitar este tipo de ataques en un módulo del kernel de Linux? ¿Qué mecanismo debería estar habilitado?

C - Ejercicio SystemD

Objetivo: Aprender algunas restricciones de seguridad que se pueden aplicar a un servicio en SystemD.

- <https://www.redhat.com/en/blog/cgroups-part-four>
- <https://www.redhat.com/en/blog/mastering-systemd>

1. Investigue los comandos:
 - a. `systemctl enable`
 - b. `systemctl disable`
 - c. `systemctl daemon-reload`
 - d. `systemctl start`
 - e. `systemctl stop`
 - f. `systemctl status`
 - g. `systemd-cgls`
 - h. `journalctl -u [unit]`

2. Investigue las siguientes opciones que se pueden configurar en una unit service de systemd:
 - a. User y Group
 - b. ProtectHome
 - c. PrivateTmp
 - d. ProtectProc
 - e. MemoryAccounting, MemoryHigh y MemoryMax
3. Tenga en cuenta para los siguientes puntos:
 - a. La configuración del servicio se instala en:
`/etc/systemd/system/insecure_service.service`
 - b. Cada vez que modifique la configuración será necesario recargar el demonio de systemd y recargar el servicio:
 - i. `systemctl daemon-reload`
 - ii. `systemctl restart insecure_service.service`
4. En el directorio `insecure_service` del repositorio de la cátedra encontrará, el binario `insecure_service`, el archivo de configuración `insecure_service.service` y el script `install.sh`.
 - a. Instale el servicio usando el script `install.sh`.
 - b. Verifique que el servicio se está ejecutando con `systemctl status`.
 - c. Verifique con qué UID se ejecuta el servicio usando `psaux | grep insecure_service`.
 - d. Abra `localhost:8080` en el navegador y explore los links provistos por este servicio.
5. Configure el servicio para que se ejecute con usuario y grupo no privilegiados (en Debian y derivados se llaman `nobody` y `nogroup`). Verifique con qué UID se ejecuta el servicio usando `ps aux | grep insecure_service`.
6. Explore el directorio `/home` y el directorio `/tmp` usando el servicio y luego:
 - a. Reconfigurelo para que no pueda visualizar el contenido de `/home` y tenga su propio `/tmp` privado.
 - b. Recargue el servicio y verifique que estas restricciones surgieron efecto.
7. Limite el acceso a información de otros procesos por parte del servicio.
8. Establezca un límite de 16M al uso de memoria del servicio e intente alocar más de esa memoria en la sección "Memoria" usando el link "[Aumentar Reserva de Memoria](#)"

D - AppArmor

9. Instale las herramientas de espacio de usuario, perfiles por defecto de apparmor y auditd (necesario para generar perfiles de forma interactiva).

```
apt install apparmor apparmor-profiles apparmor-utils auditd
```

10. Verifique si apparmor se encuentra habilitado con el comando `aa-enabled`. Si no se encuentra habilitado verifique el kernel que está ejecutando (el kernel de Debian de la VM lo trae habilitado por defecto).
11. Utilice la herramienta `aa-status` para determinar:
 - a. ¿Cuántos perfiles se encuentran cargados?
 - b. ¿Cuántos procesos y cuáles procesos de tu sistema tienen perfiles definidos?
12. Detenga y deshabilite el servicio `insecure_service` creado en la parte 1 de la práctica de forma que no vuelva a iniciarse automáticamente.

```
systemctl stop insecure_service.service  
systemctl disable insecure_service.service
```

13. Ejecute `insecure_service` manualmente usando el usuario `root` y verifique que puede acceder libremente al filesystem en <http://localhost:8080> (o la IP correspondiente donde se ejecuta el servicio).

```
/opt/sistemasoperativos/insecure_service
```

14. Generación de un nuevo profile:
 - a. Ejecutar `aa-genprof /...`
 - b. Abrir otra terminal, ejecutar `insecure_service` y navegue el sistema de archivos usando la interfaz web provista por el servicio.
 - c. Genere un perfil que permita:
 - i. Abrir conexiones tcp ipv4
 - ii. Abrir conexiones tcp ipv6
 - iii. El perfil debe incluir los siguientes perfiles (y ningún otro):
 1. `include <abstractions/base>`
 2. `include <abstractions/nameservice>`
 - iv. Listar el contenido de `/` y `/proc` pero no de otros subdirectorios de `/`
 - v. Ejecutar con los permisos del perfil actual (`mrix`) los siguientes comandos:
 1. `/usr/bin/dash`
 2. `/usr/bin/ip`
 3. `/usr/bin/mawk`
 4. `/usr/bin/ps`
15. Habilite el modo enforcing y verifique si funciona (`aa-enforcing`).
16. Si necesita volver a generar un perfil puede usar `aa-complain + aa-logprofile o` editar el profile a mano y aplicar con `apparmor_parser -r`

Ayudas:

- Es útil habilitar el modo complain y volver a ejecutar `aa-genprof` para detectar más acciones y que se agreguen al profile.

- Seguro es necesario ajustar el archivo manualmente ya que aa-genprof no siempre muestra las opciones que necesitamos.
- Verificar que no se agreguen “include” adicionales ya que traen otras reglas que van a cambiar el comportamiento.
- Para permitir acceso a un directorio:
 - /path/terminado/en/barra/ r,
- Para permitir acceso a los subdirectorios:
 - /path/terminado/en/barra/** r,
- Para denegar es lo mismo agregando deny al principio.
- Para permitir listar / pero denegar el resto:
 - / r,
 - deny /* r,
- owner se usa para acceder solo a los recursos de los cuales el proceso es owner. No lo usaremos en esta práctica.
- Siempre verificar que el perfil esté en enforce en las pruebas, si está en complain el proceso podrá acceder a todos los recursos y no estaremos probando el perfil realmente.

E - CopyFail

Nota: Para esta parte es importante seguir la práctica secuencialmente sin reiniciar la VM a menos que se pida.

1. Investigue la vulnerabilidad CopyFail (ver links en practica5/copy_fail/README.md)
 - a. ¿Es una vulnerabilidad en espacio de usuario o del kernel?
 - b. ¿Por qué el exploit abre un archivo con suid?
 - c. ¿La página modificada se almacena luego en disco?
2. Ejecute en exploit copy_fail_exp.py en la máquina virtual de la cátedra y pruebe lo siguiente:
 - a. Verifique que obtuvo root
 - b. Abra otra terminal con el usuario “so” y ejecute el comando “su” ¿Qué sucedió? ¿Por qué?
 - c. Reinicie la máquina virtual, abra una terminal con el usuario “so” y ejecute el comando “su” ¿Qué sucedió? ¿Por qué?
3. ¿Por qué se dice que puede permitir saltar el aislamiento de containers?
4. ¿Qué condiciones deben darse para que CopyFail permita afectar el sistema host si se ataca un container?
5. Laboratorio con docker compose (en la VM de la cátedra):
 - a. Compile `superuser.c` con el comando `make`. Asegúrese que el binario generado tenga propietario root y suid. El contenido del programa no es importante, No necesariamente tiene que tener alguna vulnerabilidad, podría ser cualquier binario con SUID + propietario root.
 - b. Ejecute los servicios con `docker-compose up --build -d`
 - c. Abra un shell en el container “attacker”: `docker exec -it attacker /bin/bash`
 - i. Verifique que el comando “superuser” pide una contraseña y si la contraseña no es ingresada correctamente no da permisos de root.

- ii. Ejecute el exploit “copy_fail_exp_for_docker.py”
- iii. Verifique si obtuvo root
- d. Abra un shell en victim “docker exec -it victim /bin/bash”
 - i. Ejecute “superuser”
 - ii. ¿Qué sucedió? ¿Por qué no pide password?
 - iii. Ejecute el siguiente oneliner:

```
mount /dev/sda1 /mnt/ && touch /mnt/home/so/me_escape_del_container
```

- e. Desde el usuario “so” de la VM ejecute “ls \$HOME” ¿Qué sucedió?
- 6. ¿Qué nos permite que el container “victim” se ejecute con --privileged?
- 7. ¿Por qué si el exploit se ejecutó en un container también afecta al otro container?

F - Secrets en Docker Compose (no confundir con memfd_secret)

1. Investigue los secrets de docker-compose
 - a. ¿Qué ventajas tienen sobre las env vars?
 - b. ¿Cómo se comparan con los secrets en docker swarm?
2. Verifique el archivo docker-compose.yaml en practica5/secrets
3. Ejecute los servicios con “docker-compose up --build” e:
 - a. Ejecute un shell dentro de vulnerable-insecure (docker exec -it vulnerable-insecure /bin/bash)
 - i. Ejecute el comando “env” y verifique si puede ver el los passwords y api keys
 - ii. ¿Qué procesos pueden ver esas variables de entorno?
 - b. Ejecute un shell dentro de vulnerable-secure (docker exec -it vulnerable-secure /bin/bash)
 - i. Ejecute el comando “env” y verifique si puede ver el los passwords y api keys
 - ii. Ejecute el comando mount y busque la ubicación de los secrets
 - iii. Verifique los permisos del directorio de secrets dentro del container y responda: ¿Qué procesos pueden leer los archivos de secrets?
 - iv. Investigue en <https://docs.docker.com/reference/compose-file/services/#secrets> como asignar permisos usando UGO para los secrets.

Lectura sugerida:

<http://developer.cyberark.com/blog/environment-variables-dont-keep-secrets-best-practices-for-plugging-application-credential-leaks/>